

DevOps Practices into Machine Learning

Samridhi Jain

dept. of Computer Science & Engineering
Chandigarh University
Mohali, India
samridhijain100@gmail.com

Dr. Puneet Kumar

dept. of Computer Science & Engineering
Chandigarh University
Mohali, India
professor.pkumar@gmail.com

Abstract—All industrial machine learning (ML) projects have their ultimate objective to quickly develop and deploy ML solutions. However, a lot of machine learning projects are failing, and never reach production. In order to address these concerns, this paper aims to investigate how manual ML processes can be automated and operationalised so that more ML concept can be put into production. The advantages of DevOps automation and machine learning capabilities are now being combined by industries. MLOps is the name of this combination. Our analysis found that while there are some similar aspects in defining MLOps, still there is a lack of standardisation in the term, which can occasionally lead to misunderstandings and uncertainty. We carried out the research, which includes a review of the literature and an evaluation of the MLOps tools, to fill this gap. These investigations have led us to define MLOps and identify unresolved issues in the field. In addition to this, we offer a workflow-based summary of all essential components. Finally, this paper gives guidance for applying DevOps practises to machine learning for researchers and practitioners.

Index Terms—Machine Learning, MLOps, DevOps, MLOps tools

I. INTRODUCTION

Machine learning is increasingly being used in data science to solve complicated real-world issues, revolutionise companies, and provide value in specific fields. The way decisions are made in businesses had been greatly impacted by ML. As a result, businesses can make long-term cost savings that can guarantee value for their clients [1] and also open the door to new business models.

However, many useful ML applications have not met their potential in practical contexts [2]. Numerous ML initiatives fail, and many ML proofs of concept never make it to production. From a research standpoint, this is not surprising because the ML community has concentrated heavily on creating ML models, still there is no product created that can be used for production. In order to address this issue, companies are attempting to implement DevOps principles to their ML systems [3] through the use of data scientists and operations team.

Manual ML pipeline presents distinct difficulties that must be tackled differently from those in traditional software development. ML is a metrics-driven process, and numerous rounds of experiments are run to choose the optimal model. The metrics for every trial is manually tracked, which adds to the complexity overall. It's ineffective and prone to mistakes to manually log data. No versions of the data exist. This further

increases the complexity because in order to repeat results, both code and data are needed.

Data evolves much like code does. There is no model versioning or registry available. Replicating an old model is more difficult. Developing the code is restricted to one's development environments like Jupyter because it is tough to share models and other artefacts, therefore challenging for team members to collaborate, the deployed model cannot be automatically retrained. Also because of the manual nature of the procedure, there are fewer releases and long deployment times for new models. Modifying a single parameter, such as a specific hyper-parameter that affects deep neural network's dimensions, alters the entire pipeline, makes an impact on pipeline stages that follow, and may be inconsistent or lack versioning.

In response to above mentioned difficulties, the field of machine learning operations (MLOps) was created. MLOps has attracted a lot of interest among ML practitioners due to its growth in ML applications. Despite machine learning's growth, there is not much research on MLOps because it is relatively a new concept. The aim of MLOps is to deploy and manage ML models and to code in an effective and efficient manner to accelerate development and deployment. To manage the ML life cycle, MLOps is built on DevOps techniques like CI and CD.

The remainder of the paper is organized as follows: Section II provides the literature review of the work done on MLOps. Section III describes an overview of DevOps and its components. Section IV compare MLOps with DevOps. Section V presents an overview on MLOps and typical MLOps pipeline. Section VI draws a comparison between the MLOps tools with its limitation. Section VII described the proposed MLOps workflow and Section VIII concludes our study.

II. LITERATURE SURVEY

Data collection, data extraction, and data purification are three of the most essential difficulties that must be addressed in the creation of machine learning systems in order to make ongoing improvements. Few research papers have been published to warn businesses about the pitfalls of rapid victories while constructing machine learning models.

Tamburri et al. define MLOps as the distribution of a group of software elements that carry out five ML pipeline functions [4]. A picture of the state-of-the-art in MLOps is produced

by examining developments in AI software operations. This demonstrates how MLOps is standing at the intersection of machine learning, software engineering, data engineering, and DevOps.

Zhou et al. tested the practicality of setting up an machine learning pipeline with CI/CD functionalities on different softwares with a certain hardware setup [5]. They also evaluated the performance of ML pipeline platforms, particularly Kubeflow, using multiple metrics and different models. In [5] [6], shows a highly effective technique to draw attention to the differences between the two closely related but different paradigms to make a visible difference between DevOps tools and MLOps tooling.

In order to determine how software and Data Version Control (DVC) artifacts are coupled and how their complexity evolves, an empirical analysis was done by Barrak et al. on 391 open-source projects using DVC approaches [7]. They concluded from their empirical investigation that, despite the maintenance costs, the use of DVC versioning technologies is becoming more prominent.

The different approaches, frameworks, and difficulties of Automated Machine Learning were described in [8]. The common basis of Auto machine learning frameworks is explained in terms of hyper-parameter optimization. The discussion of well-known and established frameworks for automating ML tasks also followed. The integration of ML-related automation in MLOps tasks becomes increasingly appealing as the automation of the various components of an ML pipeline matures and the framework landscape for particular challenges improves.

John et al. demonstrated that numerous firms are attempting to incorporate continuous techniques in ML system development in light of the widespread success of DevOps [9]. For complete life-cycle automation, machine learning developers and operations teams attempts to apply DevOps concepts onto ML systems. It also explores the use of DevOps in ML systems and highlights how crucial development is to MLOps.

Granlund et al. demonstrated problems with the MLOps pipeline when several organizations are included, emphasizing elements like scalability [10]. [11] This paper tries to eliminate ML-DevOps myths and misconceptions by explicitly defining what it comprises. Myths like software developers had been eliminated by data scientist. A sight of the current definition in the MLOps sector is provided by Kreuzberger et al. through literature studies and surveys, which are essential [12]. They provide us an easy source of information to determine what is known and what is unknown in a certain field of study.

Finding and tracking down every piece of research done on MLOps literature is challenging. The majority of independent studies on MLOps lack standardisation and employ dissimilar vocabularies, making meta-analysis on MLOps difficult. As a result, it is challenging to combine the findings of different investigations into a single appraisal. Additional comparable versions, such as DataOps, ModelOps, GitOps, ArchOps, DevSecOps, etc., produce too much assortment in the body of research, making it a challenge to determine whether inves-

tigations are sufficiently similar to allow for the meaningful combination of their results.

III. DEVOPS

Conventionally the developed code(patch) was launched at the release date by the developers to the operations team. The goal of DevOps, in contrast, was to close the silos in two subsidiaries: Dev and Ops. The current tendency among many firms is to automate delivery, maintaining a link between software development and software deployment.

To track changes in software artefacts and maintain consistency, DevOps uses a stack of support tools. The software development process involves the usage of many Integrated Development Environments (IDEs). Supporting tools like Git for controlling version and a Docker container for wrapping all the libraries and its dependencies are available. Codes that have been contributed by several people can be integrated using tools like Jenkins. As a result, a variety of tools like these helps in the regular and trustworthy software releases with CI/CD [13].

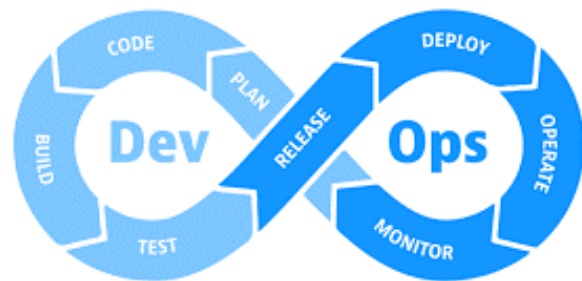


Fig. 1. DevOps Workflow

With the combination of technologies and techniques, from development to maintenance, the DevOps workflow as depicted above in Fig 1, enabling the organisation to code, build, plan, and release software fast and effectively.

One of DevOps' most important aims is to shift the culture toward greater cooperation and formation of cross-functional teams. A pipeline is a series of processes that combines the above mentioned DevOps components. No solitary generalised pipeline structure exists [14] [15]. The structure of each pipeline varies depends on the use-cases and the technology used to implement it. The pipeline elements listed below are typical in DevOps [16]:

- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Deployment
- Continuous Monitoring

IV. COMPARE MLOPS TO DEVOPS

DevOps is a collaborative effort of engineers from the development and operational disciplines to ensure a seamless transition. But with MLOps, the fundamental procedure

and objective continued to be same. However, the focus on the projects is driven by machine learning which generates additional requirements, such as management of the ML life-process, that is typically not shown by traditional DevOps [17].

DevOps focuses on optimising the delivering process and standardization of developing environments, however in MLOps, data is the major emphasis, therefore data scientists collaborate firmly with software engineers. This is another significant distinction between MLOps and DevOps [12].

V. MLOPS

Machine learning is increasingly being used in data science to solve complicated real-world issues, revolutionise companies, and provide value in specific fields. As a result, In the scientific and machine learning areas, there are several data science teams that are utilising descriptive and predictive models to try to increase total business value.

The delivery of the ML pipeline model now includes manual steps created by ML data scientists and ML operations engineering groups. Furthermore, in terms of outputs of high quality, this ML manual pipeline technique leads in significant operating expenses and delays, which can have a direct or indirect impact on a company's revenue or reputation for quality.

Since data is always changing or updating in ML applications, retraining the model is crucial. The model must be retrained to handle the newly incoming data. This retraining, or as we can say, continual training, is unique to MLOPS. This model retraining comprises automating several model validation, data validation, and retraining processes. The research indicate that the typical MLOps pipeline is as follows [9] [18] [19]:

- Continuous Integration (CI)
- Continuous Deployment (CD)
- Continuous Training (CT)

Along with these pipelines, MLOps uses several extra elements, such as feature storage and metadata management. They support features of replication and data management.

Derived from Sculley et al.'s research [20], Google Cloud suggests three MLOps process levels for constructing the CI/CD pipeline. They describe the various stages of maturity of the MLOps processes:

- Manual process
- ML pipeline automation
- CI/CD pipeline automation

VI. MLOPS TOOLS

Numerous new tools that aid in automating the ML process have surfaced in recent years. The use-case of the specific Machine Learning solution and the operations configuration determine the tools that should be used for MLOps [21]. We are not yet aware of a single tool that can build a fully automated MLOps workflow. Therefore, it's harder to automate some tasks than others.

A few commercially available products, such as MLflow, Kubeflow, and Amazon SageMaker, can provide those features

to a certain extent through automated procedures. Existing research noted the creation of numerous open-source tools, frameworks, and cloud services for MLOPS technology.

When choosing tools, it's important to consider the integration support each one provides. Some tools exclusively have Python-based APIs. Some, however, include support for JAVA and R. Another crucial element is the support for various intermediate libraries needed for model creation.

TABLE I
COMPARISON OF THREE VITAL MLOPS TOOLS

	<i>MLFlow</i>	<i>Kubeflow</i>	<i>AWS SageMaker</i>
<i>Data versioning</i>	Yes	No	Yes
<i>Pipeline versioning</i>	Yes	No	Yes
<i>Performance monitoring</i>	No	Yes	Yes
<i>Jupyter Notebook</i>	No	Yes	Yes
<i>Keras Library</i>	Yes	No	Yes

a) *MLFlow*: An ML platform called MLflow makes it possible to control the entire ML life-cycle. It offers comprehensive experiment tracking capabilities, a model registry, and a component for serving models [35]. A crucial feature of MLflow is that it can manage and run any ML library and any programming language. However, this tool has some downsides which involves the lack of built-in notebooks..

b) *Kubeflow*: End-to-end ML platform Kubeflow is built on Kubernetes [22]. Kubernetes controls each Kubeflow component, which is placed inside a container [4] [23]. Furthermore, a container is used to manage each task in an ML workflow pipeline. Along with an interactive UI and notebook, it provides extra functionality of Kubeflow pipelines but it lacks a specific tool for the CICD process.

c) *AWS SageMaker*: AWS SageMaker performs better than others, as can be shown in above table 1, but they also have flaws that needs to be fix. Despite having comparable platforms, cloud service providers are expensive and do not solve the ML issue directly through a single dashboard [23] .

Most organisations already uses various frameworks, therefore choosing a tool that is compatible with the existing platform is also crucial. Fulfilling the requirements, bringing down the complexity, and working towards automation in incremental steps should be the main objectives.

VII. PROPOSED MLOPS WORKFLOW

Machine learning pipelines principles from the DevOps architecture [20] are used to automate the process of retraining models in production using fresh data. We need a powerful automated CI/CD solution to update production pipelines quickly and reliably. The data scientists can quickly try out new suggestions in feature engineering, model architecture and hyper-parameters, thanks to the automated CI/CD infrastructure. It automate the task of creating a model, finding it in the repository, creating a deployment package, and delivering it to production environment. Fig 2 depicts the deployment of MLOps pipelines and Fig 3 depicts MLOps workflow which is utilising CI/CD, that includes automated ML pipeline setup as well as automated CI/CD processes.

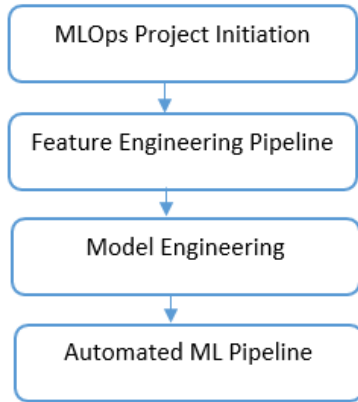


Fig. 2. Proposed MLOps Pipeline

A. GIT

Making sure that the code is properly versioned is the first step. Here GIT is chosen as version control system. The code was correctly versioned, and after accurate tagging, a stable version was always preserved in the main branch. A repository is automatically pulled when developers publish it to Github or Bit bucket. Any commit change in an SCM system leads to trigger's default setting. The pipeline will be triggered to establish new pipelines to generate the results if there is any change in the code.

B. CI/CD

The following components of CI/CD should be present: source control (SC), Machine Learning pipelines, test services, model services, feature storage, and deployment services. Building pipelines is the initial stage of the CI/CD process. To implement, these pipelines are executed in order.

According to use-cases, these pipeline should offer the necessary libraries and a supporting environment. All the required libraries are installed in a dockerfile that we have prepared. To provide the same environment in both development and production, a docker container image is produced from this dockerfile. This Docker image is crucial for maintaining uniformity in the environment.

Now push this container image into the organization's private repository or onto DockerHub, a public repository that allows for both internal and external use of the container image.

SIZE			
mlopsimg	v1	dde3a40eda15	4 days ago
2.95GB			
samridhi100/mlopsimg	v1	dde3a40eda15	4 days ago
2.95GB			

Fig. 3. samridhi100/mlopsimg Container image

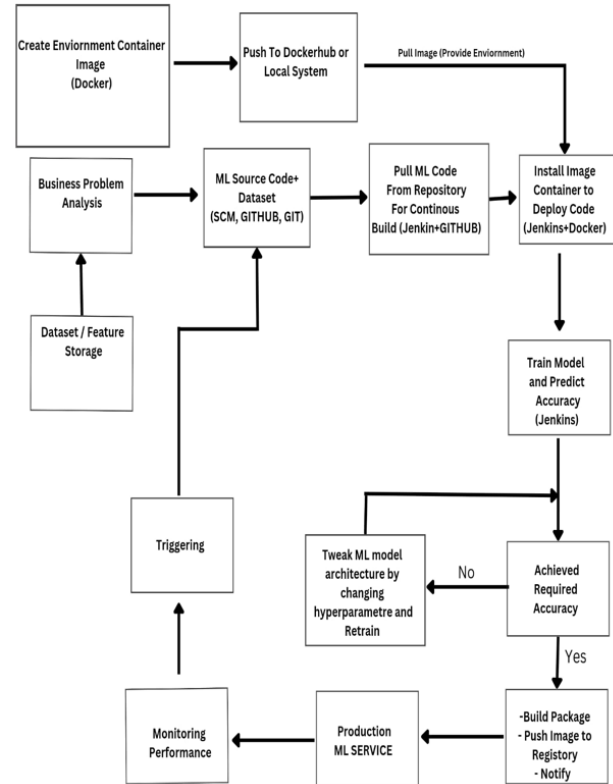


Fig. 4. Proposed MLOps Workflow

C. Jenkins

Jenkinsfile includes support for MLOps pipelines. The Docker's container with the image including all the installed software and libraries is started by Jenkins automatically. When a commit change occurs in SCM, this container pulls the machine learning code from Bitbucket or Github.

The next job in pipeline is to start training the model inside the docker container. After successful build, the container provides a metrics and logs. The CI/CD system keeps track of these metrics and logs to make it simple to duplicate and troubleshoot faults on the developer's machine using git triggers and databases. If the build metrics accuracy is less than the desired accuracy, then the machine learning model architecture gets tweak automatically. This is what we call automatic continuous training.

A notification email about the new build job is sent from the Jenkins Script whenever the job execution has been done by the pipeline, based on the build's status. After the model is successfully built in the build pipeline, this notification job is produced.

VIII. CONCLUSION

We proposed an ML automation pipeline using CI/CD concepts in this study. The strategy is built on DevOps methods, which are in charge of the integration and delivery of machine learning models that have been trained and tested. In corporate

organisations, we noticed that the manual ML technique results in significant operating expenses and delays. The suggested automated ML technique improves numerous areas, including speed to market, cross-business unit integration. Many firms finds it hard and reside at the beginning of trip while some abandon it midway owing through the problems such as aversion change, segregated teams and so on.

It is important to remember that MLOps is not a easy concept. MLOps pipelines may appear to be very automated. However, not all of the pipelines or pipeline parts are required for implementation. Depending on the implementation and use case, the pipeline and its components can be chosen.

To be successful in your MLOps journey, you must first focus on transforming your company's culture, gain senior support, select best tool-chain according to the company's requirements and automate. Future research might leverage our strategy to focus on certain features for complex models and produce helpful tools to aid in the testing of ML models.

REFERENCES

- [1] A. Miklosik, M. Kuchta, N. Evans, and S. Zak, "Towards the adoption of machine learning-based analytical tools in digital marketing," *IEEE Access*, vol. 7, pp. 85705–85718, 2019.
- [2] R. Kocielnik, S. Amershi, and P. N. Bennett, "Will you accept an imperfect ai? exploring designs for adjusting end-user expectations of ai systems," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–14, 2019.
- [3] I. Karamitsos, S. Albarhami, and C. Apostolopoulos, "Applying devops practices of continuous automation for machine learning," *Information*, vol. 11, no. 7, p. 363, 2020.
- [4] D. A. Tamburri, "Sustainable mlops: Trends and challenges," in *2020 22nd international symposium on symbolic and numeric algorithms for scientific computing (SYNASC)*, pp. 17–23, IEEE, 2020.
- [5] Y. Zhou, Y. Yu, and B. Ding, "Towards mlops: A case study of ml pipeline platform," in *2020 International conference on artificial intelligence and computer engineering (ICAICE)*, pp. 494–500, IEEE, 2020.
- [6] P. Ruf, M. Madan, C. Reich, and D. Ould-Abdeslam, "Demystifying mlops and presenting a recipe for the selection of open-source tools," *Applied Sciences*, vol. 11, no. 19, p. 8861, 2021.
- [7] A. Barrak, E. E. Eghan, and B. Adams, "On the co-evolution of ml pipelines and source code-empirical study of dvc projects," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 422–433, IEEE, 2021.
- [8] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [9] M. M. John, H. H. Olsson, and J. Bosch, "Towards mlops: A framework and maturity model," in *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 1–8, IEEE, 2021.
- [10] T. Granlund, A. Kopponen, V. Stirbu, L. Myllyaho, and T. Mikkonen, "Mlops challenges in multi-organization setup: Experiences from two real-world cases," in *2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN)*, pp. 82–88, IEEE, 2021.
- [11] S. Alla and S. K. Adari, "What is mlops?," in *Beginning MLOps with MLFlow*, pp. 79–124, Springer, 2021.
- [12] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," *arXiv preprint arXiv:2205.02302*, 2022.
- [13] M. Virmani, "Understanding devops & bridging the gap from continuous integration to continuous delivery," in *Fifth international conference on the innovative computing technology (intech 2015)*, pp. 78–82, IEEE, 2015.
- [14] M. Soni, "End to end automation on cloud with build pipeline: the case for devops in insurance industry, continuous integration, continuous testing, and continuous delivery," in *2015 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, pp. 85–89, IEEE, 2015.
- [15] T. F. Düllmann, C. Paule, and A. van Hoorn, "Exploiting devops practices for dependable and secure continuous delivery pipelines," in *2018 IEEE/ACM 4th International Workshop on Rapid Continuous Software Engineering (RCOSE)*, pp. 27–30, IEEE, 2018.
- [16] M. Gokarna and R. Singh, "Devops: a historical review and future works," in *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, pp. 366–371, IEEE, 2021.
- [17] M. Treveil, N. Omont, C. Stenac, K. Lefevre, D. Phan, J. Zentici, A. Lavoillotte, M. Miyazaki, and L. Heidmann, *Introducing MLOps*. O'Reilly Media, 2020.
- [18] E. Raj, D. Buffoni, M. Westerlund, and K. Ahola, "Edge mlops: An automation framework for aiot applications," in *2021 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 191–200, IEEE, 2021.
- [19] R. Subramanya, S. Sierla, and V. Vyatkin, "From devops to mlops: Overview and application to electricity market forecasting," *Applied Sciences*, vol. 12, no. 19, p. 9851, 2022.
- [20] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in neural information processing systems*, vol. 28, 2015.
- [21] E. Raj, *Engineering MLOps: Rapidly build, test, and manage production-ready machine learning life cycles at scale*. Packt Publishing Ltd, 2021.
- [22] A. M. Domenech and A. Guillén, "ml-experiment: A python framework for reproducible data science," in *Journal of Physics: Conference Series*, vol. 1603, p. 012025, IOP Publishing, 2020.
- [23] A. Posoldova, "Machine learning pipelines: From research to production," *IEEE Potentials*, vol. 39, no. 6, pp. 38–42, 2020.